Zonoth

SESSION 09 · 2H

Operating Kubernetes

A repeatable troubleshooting method, the numbers that tell you what is happening, and the objects that keep Pinboard alive and contained.

Troubleshooting method | Observability | Autoscaling | Security basics | Lab 09

SESSION 09

What we'll cover today

1 · Troubleshoot & observe

One method — get → describe → logs → events → exec/debug — the status zoo, `kubectl debug`, `kubectl top`, k9s & Headlamp.

2 · Scale & protect

HPA `autoscaling/v2`, PodDisruptionBudget, ResourceQuota & LimitRange, RBAC, securityContext and Pod Security Admission.

3 · Demos

Break pinboard-api three ways and fix it live with the method; then watch an HPA react to `/api/burn`.

4 · Lab 09

Operate Pinboard: metrics-server, an HPA with a load generator, debugging a broken API, a PDB and the `restricted` PSA label.

01

Troubleshooting & observability

Five commands, always in the same order. Panic is what happens when you don't have a method.

THE METHOD

Always in this order

01 — get
What is the cluster's opinion? Status, restarts, age

→

02 — describe
Why that status — spec, conditions, Events at the bottom

→

03 — logs
What the app itself said, incl. `--previous`

→

04 — events
Namespace-wide, sorted by time — scheduler & kubelet

→

05 — exec / debug
Go inside: shell, ephemeral container, node

> Steps 1–2 answer "is it Kubernetes?"; step 3 answers "is it the app?". Never start at step 5 — it is the slowest and the most misleading.

STEP 01

get — the cluster's own opinion

- `-o wide` adds node and Pod IP — often the whole answer
- RESTARTS with an age in brackets means a crash loop, not history
- Query by label, never by remembered Pod name
- Check the **chain**: Deployment → ReplicaSet → Pod → Endpoints

> An empty `ENDPOINTS` column on a Service means no **ready** Pod matches its selector.

BASH

```
kubectl -n pinboard get pods -o wide
kubectl -n pinboard get deploy,rs,svc,endpointslice

# by label, not by name
kubectl -n pinboard get pods \
  -l app.kubernetes.io/name=pinboard-api

# the whole object, as the API server sees it
kubectl -n pinboard get pod pinboard-api-7d9f4-2xk9 \
  -o yaml

# every namespace, when you don't know where it is
kubectl get pods -A --field-selector \
  status.phase!=Running
```

STEP 02

describe — read from the bottom up

```text
$ kubectl -n pinboard describe pod pinboard-api-7d9f4-2xk9
  State:          Waiting
    Reason:       CrashLoopBackOff
  Last State:     Terminated
    Reason:       Error          Exit Code: 1
  Restart Count:  5
Events:
  Type     Reason     Age                From      Message
  ----     ------     ----               ----      -------
  Normal   Scheduled  3m                 default-scheduler  Successfully assigned …
  Normal   Pulled     2m (x4 over 3m)    kubelet   Container image "pinboard-api:1.2-broken"
  Warning  BackOff    30s (x8 over 3m)   kubelet   Back-off restarting failed container api
```

- **State / Last State** tell you what is happening **now** and what happened last
- **Events** are the kubelet and scheduler talking — and they expire after ~1 hour

STEP 03

logs — what the application said

- `--previous` reads the **dead** container — the only way to see why it crashed
- Address a Deployment, not a Pod, and kubectl picks one for you
- `-l` plus `--all-containers` tails a whole tier at once
- Pinboard: set `LOG_FORMAT=json` so a log stack can parse it

BASH

```
kubectl -n pinboard logs -f deploy/pinboard-api

# the container that already died
kubectl -n pinboard logs pinboard-api-7d9f4-2xk9 \
  --previous

# a whole tier, last 20 lines each
kubectl -n pinboard logs --tail=20 --all-containers \
  -l app.kubernetes.io/name=pinboard-api

kubectl -n pinboard logs deploy/pinboard-api \
  --since=10m --timestamps
```

STEP 04

events — the namespace timeline

```bash
 kubectl -n pinboard get events --sort-by=.lastTimestamp

kubectl -n pinboard get events --field-selector type=Warning
kubectl -n pinboard get events --watch

# events for one object only
kubectl -n pinboard get events \
  --field-selector involvedObject.name=pinboard-api-7d9f4-2xk9
```

- `get events` is **not** sorted by default — always pass `--sort-by=.lastTimestamp`
- Scheduling failures, image pulls, probe failures, evictions and OOM kills all land here
- Events belong to the namespace: use them when you don't yet know which Pod is guilty

PATTERN RECOGNITION

The status zoo

STATUS	WHAT IT MEANS	FIRST THING TO CHECK
Pending	Not bound to a node yet	`describe` → Events: insufficient cpu/memory, taints, or an unbound PVC
ImagePullBackOff	The image can't be fetched	Tag typo, private registry auth, or `kind load docker-image` never ran
CrashLoopBackOff	Container starts, exits, repeats	`logs --previous` — here: `CRASH_ON_START=true` in the env
OOMKilled	Exceeded its memory limit	`describe` → Last State, exit 137. Raise `limits.memory` or fix the leak
Evicted	Node under disk/memory pressure	`describe node` → Conditions; set honest `requests` to get a better QoS class
Unschedulable	No node satisfies the Pod	nodeSelector / affinity / taints, or the cluster is simply full

STEP 05

exec & kubectl debug

BASH

```bash
  # pinboard-api is distroless — there is no shell to exec into
kubectl -n pinboard exec -it deploy/pinboard-api -- sh    # "sh": not found in $PATH

# 1 · ephemeral container: attach busybox to the RUNNING Pod, no restart
kubectl -n pinboard debug -it pinboard-api-7d9f4-2xk9 \
  --image=busybox --target=api -- sh

# 2 · a copy of the Pod with a shell, for something that crashes on start
kubectl -n pinboard debug pinboard-api-7d9f4-2xk9 --copy-to=api-debug \
  --image=busybox --share-processes -it -- sh

# 3 · a shell on the node itself; the node root is mounted at /host
kubectl debug node/pinboard-worker -it --image=busybox
```

⚠ `--target=api` shares the process namespace of that container. Without it you only share the network and volumes.

LIVE DEMO

Break pinboard-api three ways — and fix it with the method

A bad image tag, `CRASH_ON_START=true`, and a 16 Mi memory limit. Same five commands each time, out loud.

What you'll see · ImagePullBackOff → CrashLoopBackOff → OOMKilled

Files · demos/session06/api-deploy-v1.yaml

Duration · 10 min

NUMBERS

metrics-server & kubectl top

```bash
 kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/\
releases/latest/download/components.yaml

# kind only: the kubelet serves a self-signed serving certificate
kubectl -n kube-system patch deploy metrics-server --type=json -p='[{"op":"add",
  "path":"/spec/template/spec/containers/0/args/-","value":"--kubelet-insecure-tls"}]'

kubectl top nodes
kubectl -n pinboard top pods --containers --sort-by=cpu
```

- metrics-server scrapes the kubelet every 15 s and keeps **only the latest value** in memory
- It is not monitoring — no history, no alerts. It exists to serve `top` and the **HPA**
- No metrics-server → `kubectl top` fails and every HPA reports `<unknown>`

TOOLING

k9s and Headlamp

k9s — the terminal cockpit

A single binary. `:pods`, `:deploy`, `l` for logs, `d` for describe, `s` for a shell, `Ctrl-D` to delete. It is the five-step method with fewer keystrokes.

Headlamp — the web UI

CNCF-hosted, extensible, runs locally or in-cluster with proper RBAC. This is what replaced the old Kubernetes Dashboard in practice.

⚠ The legacy **Kubernetes Dashboard** is Helm-only now and is not the tool to teach in 2026. Whatever the UI, it is still just the API — everything it does, `kubectl` can do.

BEYOND ONE CLUSTER

When kubectl logs is not enough

Logs

Ship stdout with a DaemonSet agent (Promtail/Fluent Bit) into **Loki** or **ELK/OpenSearch**. Survives Pod deletion; queryable by label.

Metrics

Prometheus scrapes and stores history; Grafana draws it; Alertmanager wakes someone. metrics-server is only the live value.

Traces

OpenTelemetry is the vendor-neutral standard: one SDK and one Collector for logs, metrics and traces.

Your side of the contract is simple: log to **stdout** as structured JSON (`LOG_FORMAT=json`) and never write log files inside the container.

02

Scaling, quotas and security

The objects that let you hand a namespace to a team and sleep at night.

HorizontalPodAutoscaler — autoscaling/v2

- Scales **replicas**, never the Pod's own size
- Utilisation is a percentage of `requests.cpu`: **no requests, no HPA**
- v2 also takes memory, Pod and external metrics
- Never set `replicas:` in a Deployment that an HPA owns

YAML

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: {name: pinboard-api, namespace: pinboard}
spec:
  scaleTargetRef: {apiVersion: apps/v1, kind: Deployment,
                   name: pinboard-api}
  minReplicas: 2
  maxReplicas: 6
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: {type: Utilization, averageUtilization: 50}
```

HOW IT DECIDES

The formula — and behaviour

```text
 desiredReplicas = ceil( currentReplicas × currentMetric / targetMetric )

3 replicas at 90 % CPU, target 50 %   →  ceil(3 × 90/50) = ceil(5.4) = 6
6 replicas at 20 % CPU, target 50 %   →  ceil(6 × 20/50) = ceil(2.4) = 3   (after the window)
```

```yaml
  behavior:
  scaleUp:
    policies: [{type: Pods, value: 4, periodSeconds: 15}]
  scaleDown:
    stabilizationWindowSeconds: 60     # look back 60 s and take the highest
    policies: [{type: Percent, value: 50, periodSeconds: 60}]
```

- A ±10 % tolerance stops it reacting to noise; the loop runs every 15 s
- Scale **up** fast, scale **down** slowly — that is what the window is for

THE OTHER TWO AXES

VPA, cluster autoscaler, Karpenter

VPA — bigger Pods

Recommends or sets `requests` from observed usage. Run it in `recommender` mode next to an HPA; both on CPU at once will fight.

Cluster Autoscaler — more nodes

Watches for Pending Pods that don't fit and grows the node group; drains and removes underused nodes.

Karpenter — better nodes

Picks an instance type per pending workload instead of scaling fixed groups. The modern default on AWS, spreading elsewhere.

HPA needs spare node capacity to be useful. On kind there is none to add — which is why the lab caps the HPA at 6 tiny replicas.

VOLUNTARY DISRUPTION

PodDisruptionBudget

- Protects against **voluntary** disruption: `kubectl drain`, node upgrades, the cluster autoscaler
- It does **not** protect against a crash, an OOM kill or a node dying
- `minAvailable` or `maxUnavailable` — never both
- A PDB that can never be satisfied blocks node drains forever

YAML

```yaml
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: pinboard-api
  namespace: pinboard
spec:
  minAvailable: 2          # or maxUnavailable: 1
  selector:
    matchLabels:
      app.kubernetes.io/name: pinboard-api
```

⚠ `minAvailable: 2` with `replicas: 2` means a drain can never evict anything.

NAMESPACE GUARDRAILS

ResourceQuota & LimitRange

- **ResourceQuota** caps the whole namespace — the sum of all requests and limits, and object counts
- With a quota on cpu/memory, a Pod **without** requests is rejected outright
- **LimitRange** injects defaults and enforces min/max per container
- Together: nobody can starve the cluster, and nobody has to remember to type requests

YAML

```yaml
  kind: ResourceQuota        # apiVersion: v1
metadata: {name: pinboard-quota, namespace: pinboard}
spec:
  hard:
    requests.cpu: "2"
    requests.memory: 2Gi
    pods: "20"
---
kind: LimitRange           # apiVersion: v1
metadata: {name: pinboard-defaults, namespace: pinboard}
spec:
  limits:
    - type: Container
      defaultRequest: {cpu: 50m, memory: 32Mi}
```

LIVE DEMO

Watch the HPA react to `/api/burn`

A load generator hammers the CPU-burn endpoint while `kubectl get hpa --watch` and `kubectl top pods` run side by side.

What you'll see · 3 → 6 replicas, then a 60 s scale-down window

Files · demos/session09/hpa.yaml · load-generator.yaml

Duration · 6 min

RBAC in five minutes

- **Subject** — a ServiceAccount (in-cluster) or a user/group (from the auth provider)
- **Role** = verbs on resources **in one namespace**; **ClusterRole** = cluster-wide or reusable
- Purely additive: there is no "deny" rule

YAML

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: pinboard
rules:
  - apiGroups: [""]
    resources: ["pods", "pods/log"]
    verbs: ["get", "list", "watch"]
```

RoleBinding: gluing the two together

- **RoleBinding** glues a subject to a Role
- It may also bind a **ClusterRole** — still scoped to that namespace
- Test with `kubectl auth can-i`, adding `--as` to impersonate the ServiceAccount

YAML

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
  namespace: pinboard
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: pod-reader
subjects:
  - kind: ServiceAccount
    name: pinboard-reader
    namespace: pinboard
```

securityContext — the four settings that matter

- `runAsNonRoot` — refuse to start an image that runs as UID 0
- `readOnlyRootFilesystem` — mount an `emptyDir` anywhere it must write
- `capabilities.drop: ["ALL"]` — no NET_ADMIN, no CHOWN, nothing
- `seccompProfile: RuntimeDefault` — the runtime's syscall filter, on by choice
- Plus `allowPrivilegeEscalation: false` — no setuid surprises

YAML

```yaml
spec:
securityContext:                # Pod-level
  runAsNonRoot: true
  seccompProfile:
    type: RuntimeDefault
containers:
  - name: api
    securityContext:            # container-level wins
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop: ["ALL"]
```

ENFORCEMENT

Pod Security Admission

- **privileged** — no restrictions; for system namespaces only
- **baseline** — blocks the well-known escapes: hostPath, hostNetwork, privileged
- **restricted** — also demands non-root, dropped capabilities and RuntimeDefault seccomp
- Three modes per level: `enforce`, `audit`, `warn` — roll out with warn first

YAML

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: pinboard
  labels:
    app.kubernetes.io/part-of: pinboard
    pod-security.kubernetes.io/enforce: restricted
    pod-security.kubernetes.io/warn: restricted
```

It is just labels on the Namespace — and it rejects the Pod at **admission**, before anything runs.

LAB 09 · 30–40 MIN

Operate Pinboard

Install metrics-server on kind, put an HPA in front of `pinboard-api` and drive it with a load generator hitting `/api/burn`. Then break the API twice — `CRASH_ON_START` and a 16 Mi memory limit — and fix it with the five-step method before locking the namespace down.

You'll practise: `kubectl top` · HPA v2 (cpu 50 %, 2–6 replicas) · watching a scale-up and a scale-down · `logs --previous` · `kubectl debug --target=api` · a PodDisruptionBudget · `securityContext` · the `restricted` PSA label

Key takeaways

✓ One method, every time

get → describe → logs → events → exec/debug. The status name tells you where to jump in.

✓ No requests, no autoscaling

HPA v2 works in percentages of `requests.cpu`, and `behavior` decides how violently it reacts.

✓ Guardrails are cheap

PDB, ResourceQuota, LimitRange, RBAC and a securityContext are a dozen lines that prevent whole classes of incident.

✓ Next up

Session 10 — the production-ready checklist, Helm and Kustomize, and the 80-minute capstone.

Zonoth

Questions?

Up next — Session 10: Capstone — ship Pinboard like production. Bring your cluster.